

Allowing contract predicates on non-first declarations

Abstract

This paper proposes allowing contract predicates on non-first declarations. The rationale of doing so is, in a nutshell, to allow preconditions and postconditions to be implementation details.

Rationale

There are users that do not want to expose a contract checking mechanism in their class interfaces; there may be an English contract, but the language-level contract enforcement mechanism is an implementation detail that function declarations in a class definition should not expose.

It is probable that some would suggest that such use cases could be handled with contract assertions in function bodies. The reason why this paper proposes allowing contract predicates on non-first declarations is that

- In some cases, there is no body to put an assertion in; defaulted functions and pure virtual functions are examples of this.
- For postconditions, postconditions are guaranteed to be checked (when checking is enabled in the first place) at function exit; relying on contract assertions is more error-prone.

Usage examples

We basically propose being able to do this:

```
struct X
{
    void f();
};

void X::f() [[expects: foo]]
{
    ...
}
```

Implementation impact, semantic restrictions

Allowing contract predicates on a non-first declaration probably means that in some such cases, the checking code can't be laid down at the call site unless the definition is also seen by the compiler.

There has been a suggestion that this ability should be modeled after either default arguments or after inline, but neither solution seems really necessary, because

- the known use-cases are such that the contract predicates are desirable on a definition without having been written on a previous declaration.
- there doesn't seem to be a use case for having contracts on a non-defining declaration.

However, having said that, the following wordsmithery merely lifts the restriction that the first declaration must have the contract conditions, and allows the contract conditions to be placed on a defining declaration that is not necessarily the first declaration.

Wording

In [dcl.attr.contract.cond]/1, modify as follows:

A contract condition is a precondition or a postcondition.
Either the first declaration or a defining declaration of a function shall specify all contract conditions (if any) of the function.
Subsequent declarations shall either specify no contract conditions or the same list of contract conditions;
no diagnostic is required if corresponding conditions will always evaluate to the same value.
The list of contract conditions of a function shall be the same if the non-defining declarations of that function appear in different translation units;
no diagnostic required.